Towards the Extension of FPG-AI Toolflow to RNN Deployment on FPGAs for On-board Satellite Applications

Tommaso Pacini*, Emilio Rapuano*, Luciano Tuttobene*, Pietro Nannipieri*, Luca Fanucci*, Silvia Moranti†
*Department of Information Engineering, University of Pisa, Pisa, Italy
†European Space Research and Technology Centre, European Space Agency (ESA), Noordwijk, The Netherlands

***Abstract*—In recent years, research in the space community has shown a growing interest in the deployment of Deep Neural Networks on-board satellites, mostly driven by system miniaturization and commercial competition. Field Programmable Gate Arrays (FPGAs) have proven to be competitive accelerators for these algorithms and works proposing methods for automating the design on these devices have acquired relevance. The common purpose is to enable a wide range of users without specific skills to accelerate DNN models on FPGA with reduced development times. This paper describes the process of improving FPG-AI, a novel technology-independent toolflow that efficiently automates DNN deployment on FPGAs belonging to different vendors and with desperate resource budgets. The proposed techniques are meant to be a first step towards the extension of FPG-AI to Recurrent Neural Networks (RNNs), a subset of DNNs designed for processing sequences of data. As a first step, we discuss model compression strategies for enabling energy-efficient hardware acceleration of Gated Recurrent Units (GRUs), one of the most popular RNN layers. The article then presents a GRU-based hardware accelerator designed to be compliant with the identified quantization algorithm. The architecture features an optimized dataflow to enable hardware reuse over time and maximize resource efficiency without affecting the performance of the architecture. Preliminary results are reported for a state-of-the-art RNN-based model for Fault Detection Isolation and Recovery (FDIR) onboard the satellite to evaluate the proposed methods. A comparative analysis is also performed with the NVIDIA Jetson Nano Embedded GPU to characterize our accelerator with respect to a state-of-the-art platform for RNN inference.**

***Index Terms*—FPGA, RNN, FDIR**

I. Introduction

Nowadays, the deployment of Artificial Intelligence (AI) on-board spacecrafts has become one of the main topics addressed by both research and industry. One of the reasons behind this interest is the high potential demonstrated by Deep Neural Networks (DNNs) for several space applications, such as object-detection and recognition [1], image scene classification [2], agricultural-crop detection [3], and change detection [4], outperforming classical approaches in terms of performance and time to design.

An open research point regards the choice of hardware accelerators for DNNs, which are computationally intensive and memory-hungry algorithms. Their deployment on board spacecrafts requires finding acceptable design trade-offs for the reduced power and memory budget of these systems, where power generation and heat dissipation are stringent design issues. Field Programmable Gate Arrays (FPGAs) are a good candidate for the acceleration of DNNs for on-the-edge space applications thanks to their energy efficiency and space-qualified technology [5]. Nonetheless, the large amount of parameters and operations involved in DNNs often collide with the resource scarcity of FPGAs and make the implementation process a challenging task [6]. To facilitate and accelerate the experimentation and development of such networks on FPGAs, several automation tooflows are being released [7]. The common purpose is to enable a wide range of users without specific expertise to efficiently accelerate DNNs on FPGAs with competitive development times.

Among the emerging frameworks, FPG-AI [8] constitutes a valuable option for promoting the deployment of AI onboard the satellite. The latter is a novel technology-independent toolflow for automating the deployment of Convolutional Neural Networks (CNNs) on FPGA. The framework is a ready-to-use tool that offers to the user the possibility to efficiently map CNNs on FPGAs from different vendors and with heterogeneous resource budgets, also enabling fine-grain control over the final design in terms of area and performance. At present, FPG-AI does not provide support for Recurrent Neural Network (RNNs), Deep Learning (DL) models that are being evaluated for a variety of space applications. One of the most prominent areas of use is the control of satellites, where RNNs can be exploited to model the dynamics of a spacecraft and predict its behavior, allowing for more precise control and maneuvering [9]. Other applications involve anomaly detection [10], autonomous landing [11], and data analysis.

In this article, we present and discuss design techniques meant to be a first step toward the support of FPG-AI to RNNs. We propose an end-to-end flow to implement on FPGA one of the most used RNN layers, the Gated Recurrent Unit (GRU), ranging from model compression techniques to hardware architecture. Extending the set of supported models will increase the range and type of applications that the toolflow can target, thus broadening the pool of users. The main contributions of this work include:

- Definition and analysis of an end-to-end flow to efficiently accelerate GRUs on FPGA technology, discussing

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 10,2026 at 19:22:59 UTC from IEEE Xplore. Restrictions apply.

in detail the impact of quantization strategies and proposing an optimized dataflow scheduling for achieving high resource efficiency.
- Evaluation of the proposed methods for a space-oriented Fault Detection Isolation and Recovery (FDIR) case study trained to forecast housekeeping signals coming from Mars Advanced Radar for Subsurface And Ionosphere Sounding (MARSIS), thus enabling the first characterization of the target model on space-qualified technology.
- Comparison of the obtained results with an Nvidia Jetson Nano embedded Graphics Processing Unit (GPU) to provide a benchmarking.

The article outline is the following: Section II reports a concise overview of FPG-AI, Section III-A describes the design choices behind model compression and Section III-B discusses the developed hardware topology. Section IV shows the results achieved on the considered case study together with a comparison with a commercial edge device, the Nvidia Jetson Nano. Finally, Section V offers the conclusions of our work.

II. FPG-AI Overview

In this section, we summarize the design flow of FPG-AI, a novel toolflow for automating the acceleration of CNNs onboard FPGAs. FPG-AI receives as input a pre-trained CNN, the application dataset, and the target FPGA device. As a first step, the toolflow applies model compression techniques to prepare the target network for the hardware acceleration. In this phase, a post-training quantization algorithm converts the model from single-precision Floating-point (FP) to Fixed-point (FXP) arithmetic. A Design Space Exploration (DSE) process receives the compressed model and explores the parameter space defined by the underlying architecture. The hardware architecture is the Modular Deep Learning Engine (MDE), a fully handcrafted Hardware Description Languages (HDL)-based design that is portable on devices belonging to different vendors and with disparate resource budgets. FPG-AI generates as output an accelerator customized for the given CNN-FPGA pair that respects the additional constraints received by the user. The toolflow provides the HDL sources of the accelerator and not the final bitstream, allowing the user to exploit the unused portion of the FPGA for complementary tasks and enabling space qualification activities.

III. RNN-to-FPGA Design Flow

In this section, we describe the proposed RNN-to-FPGA end-to-end flow to be integrated within FPG-AI. We present a novel hardware accelerator built on the model compression techniques presented in [12], which are briefly described to fully understand the architecture topology.

A. Model Compression

The quantization process takes as input a pre-trained single-precision FP RNN and gives as output the quantized FXP version of that model. More precisely, uniform-symmetric

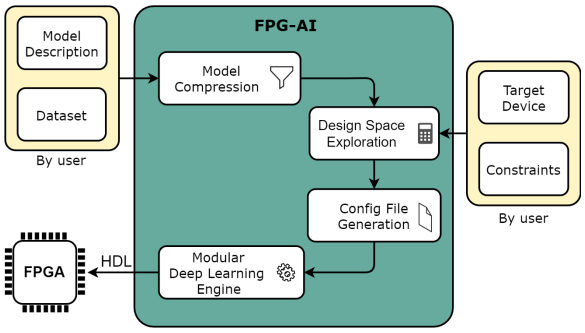


Fig. 1. FPG-AI toolflow

quantization is used to convert each FP value x into its integer version x_{int}, as shown in Equation 1:

$$x_{int} = round\left(\frac{x}{LSB_x}\right) \tag{1}$$

LSB_x is the value to be associated with the least significant bit for the two's complement (C2) representation of the integer x_{int}, which will be processed by the hardware.
For recurrent layers, the presence of closed-loop paths requires some feedback signals (i.e. cell state c_t or cell output h_t) to be modified before re-entering the layer. In the particular case of all LSB values being a power of two, and with the hypothesis of LSB values becoming smaller going from the input to the output of the cell, this loop operation can simply consist in a truncation applied on the fixed-point representation of the feedback signal (i.e. cutting out a certain amount of bits from the right side of the C2 string). By executing the truncation operation, the LSB value of a fixed-point number changes as shown in Equation 2:

$$LSB_{trunc} = LSB_x \cdot 2^{b_x} \tag{2}$$

where LSB_x is the LSB value before truncation and b_x determines the number of bits to be truncated. Truncation is also applied in other points of the network, for example at the output of point-wise multipliers, to avoid excessively small LSBs and thus further reduce the memory footprint with negligible effects on application metrics. Truncation is a very simple operation to be performed by a custom hardware accelerator designed for FPGAs, bringing advantages in terms of resource utilization and power consumption.
Figure 2 shows the model of the GRU cell with the inclusion of truncation operations. The pale blue blocks constitute the three separate gates of the GRU layer, namely the Reset gate (r_t), the Update gate (z_t), and the Output gate (h'_t). These units are divided into two categories depending on the activation function applied: tanh or sigmoid (indicated with the symbol σ). Each gate has its own weight matrices (U and W) and bias (b). U and W are respectively multiplied (through matrix-vector scalar product) with the current input vector x_t concatenated either with the cell state from the previous timestep h_{t-1} or with the output of the leftmost pointwise multiplier.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 10,2026 at 19:22:59 UTC from IEEE Xplore. Restrictions apply.

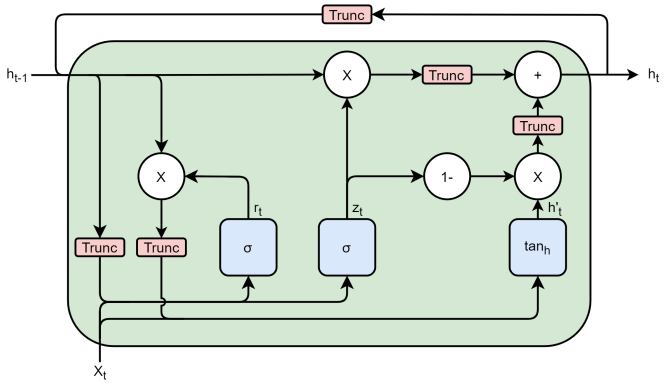


Fig. 2. Quantized GRU layer

Conversely, white blocks are associated with sum and product pointwise operations. The pale red blocks indicate the points where truncation operations are inserted by the quantization algorithm.

B. Hardware Architecture

The proposed hardware architecture is reported in Figure 3.

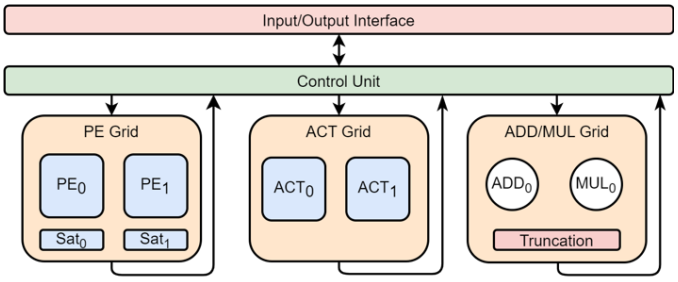


Fig. 3. Hardware accelerator

The main building blocks for the computations are:

- Control Unit (CU): this unit schedules the sequence of operations and monitors the dataflow for implementing resource arbitrage.
- Processing Element (PE): this block is responsible for the multiply and accumulate operations within the gate. The number of instantiated PEs is equal to the maximum number of gates working in parallel, which is 2. The Saturation block discards a configurable number of left-most bits to re-scale the dynamic within [-6,6] and further reduce the signal bitwidths. Indeed, the outputs of the PEs are forwarded to the Sigmoid and Tanh functions, whose trend is nearly constant outside this range.
- Activation (ACT): this unit implements Sigmoid and Tanh activation functions exploiting piece-wise linear approximations. Therefore, it requires a comparator to evaluate the thresholds, and a multiplier and an adder to compute the value of the approximated function. The number of ACTs is equal to the number of PEs.
- Point-wise ADD/MUL: these components are responsible for point-wise addition and multiplication. The Truncation block discards a configurable number of right-most bits to avoid excessively small LSBs and further reduce memory footprint.

By analyzing the dataflow of the cell, we observe that the Output gate never works simultaneously with the Reset and Update gate. For this reason, we defined a resource-sharing policy to enable hardware reuse over time and maximize resource efficiency. Figure 4 reports the implemented dataflow to execute the operation within the GRU layer.

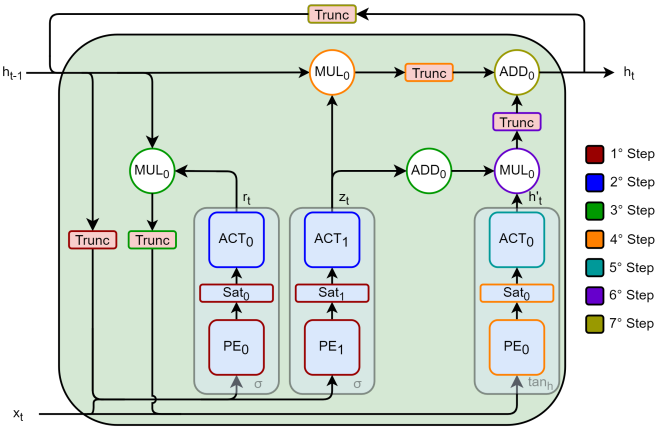


Fig. 4. Implemented dataflow and resource-sharing strategy

Layer computation is performed through several consecutive steps, each of which uses a predefined set of resources. As shown in the figure, this approach enables the reuse of the following blocks: PE_0, ACT_0, Sat_0, MUL_0, and ADD_0. This scheduling allows to minimize area occupation and achieve high levels of resource efficiency without affecting the throughput of the system. Indeed, a hardware accelerator that does not implement resource-sharing can not outperform our solution due to the wait times introduced by the feedback loop.

IV. Results

We evaluated the proposed techniques for a state-of-the-art RNN-based FDIR application [13] that forecasts housekeeping signals coming from MARSIS, which was launched with European Space Agency (ESA)'s mission Mars Express. The RNN is used to predict the evolution of the system, receiving as input temperature measurements and generating as output predictions on future values. Two separate models are provided, namely 2GRU and stacked2GRU, both exploiting GRU layers and offering different trade-offs in terms of model size (respectively 3.8K and 38K parameters) and Mean Absolute Error (MAE). The latter is the application metric used to evaluate the trained RNN models. To correctly implement anomaly detection, MAE values under the 0.1 threshold are required. Figure 5 reports the topology of the two models.

Table I reports the quantization settings for each model together with the corresponding MAE, memory footprints, and

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 10,2026 at 19:22:59 UTC from IEEE Xplore. Restrictions apply.

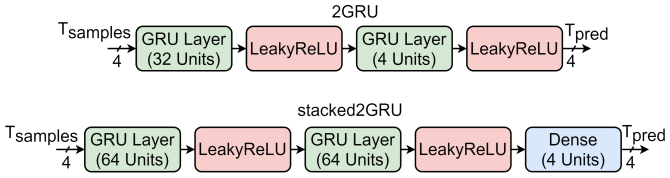


Fig. 5. Architecture of 2GRU and stacked2GRU models

compression rate. The memory footprints have been computed considering the number of bits necessary for representing the weights of both the original FP model and the compressed FXP model. The compression rate is defined as the percentage reduction in memory footprint when quantizing from FP to FXP arithmetic.

TABLE I
SUMMARY OF QUANTIZATION RESULTS

Model Architecture	2GRU	stacked2GRU
FP MAE	0.064793	0.046151
FXP MAE	0.076331	0.054003
Quantization Error	0.011538	0.007852
FP Memory Footprint [Mb]	0.117	1.16
FXP Memory Footprint [Mb]	0.018	0.145
Compression Rate	84.5%	87.5%

We implemented the obtained accelerators on a Xilinx Kintex UltraScale XQRKU060, a radiation-tolerant FPGA, thus enabling a first characterization of the target case study on space-qualified technology. Table II reports the resource utilization of both 2GRU and stacked2GRU models on the target device.

TABLE II
IMPLEMENTATION RESULTS ON XILINX KINTEX ULTRASCALE KU060

Model Architecture	**LUT**	**FF**	**LUTRAM**	**BRAM**	**DSP**	**Frequency [MHz]**
2GRU	28390 (8.56%)	10088 (1.52%)	0 (0%)	0 (0%)	354 (12.83%)	86.90
stacked2GRU	92482 (27.88%)	42567 (6.42%)	0 (0%)	0 (0%)	1170 (42.39%)	72.46

To provide a term of comparison and evaluate the obtained results, we ran both the 2GRU and stacked2GRU models on an embedded Nvidia Jetson Nano. The latter is a portable System-on-a-Chip (SoC) composed of power-efficient ARM Central Processing Unit (CPU) and NVIDIA GPU designed for embedded systems that require GPU-friendly computations. As other Commercial Off The Shelf (COTS) solutions, the Nvidia Jetson Nano is being examined as a solution for the very high demands for real-time and low-latency processing for non critical applications onboard low-cost satellites as CubeSats. The device can operate at a wattage between 5W - 10W (MAXN). We measured the average power consumption during inference using the command-line tegrastats utility. The models have been deployed exploiting the TensorflowLite toolkit distributed within the TensorFlow 2.0 framework exploiting single-precision FP arithmetic. Table III reports the comparison between FPGA and COTS acceleration in terms of power consumption, inference time, energy per inference, Total Ionizing Dose (TID), and Single Event Latch-up (SEL) metrics.

TABLE III
COMPARISON WITH JETSON NANO

		Kintex US XQRKU060	**Jetson Nano (5W)**	**Jetson Nano (MAXN)**
Mean Absolute Error	2GRU	0.076331	0.064793	0.064793
	stacked2GRU	0.054003	0.046151	0.046151
Power [W]	2GRU	0.728	2.522	2.918
	stacked2GRU	0.874	2.549	3.121
Inference Time [μs]	2GRU	4.56	833.72	510.74
	stacked2GRU	15.87	1409.25	863.26
Energy per Inference [μJ]	2GRU	3.32	2102.6	1490.3
	stacked2GRU	13.87	3592.2	2694.2
TID [Krad]		100	20	20
SEL [MeV·cm^2/mg]		80	N/A	N/A

The reported metrics show that the developed FPGA-based accelerators constitute a valuable solution for all the considered metrics. This result is achieved thanks to the specificity of the FPGA designs, which are customized for the FPGA-RNN pair taken into consideration.

V. CONCLUSION

This article presents design techniques for extending the set of supported layers of FPG-AI, a novel technology-independent CNN-to-FPGA toolflow. Contrary to other state-of-the-art solutions, this framework enables the deployment of CNNs onboard devices belonging to different vendors and with desperate resource budgets. Thanks to this feature, FPG-AI is meant to be the first step toward the deployment of DNNs onboard devices not yet supported by any framework, as in the case of NanoXplore FPGAs or recently released components. We propose and discuss an end-to-end flow to implement RNNs on FPGA technology, ranging from model compression to hardware acceleration. To evaluate the proposed methods, we accelerated a RNN-based FDIR application trained to forecast housekeeping signals coming from MARSIS, which was launched with ESA's mission Mars Express. The application of the model compression techniques led to am 84.5% compression rate with negligible effects on the application metric. After tuning the hardware architecture, we deployed the accelerator on a Xilinx Radiation Tolerant Ultrascale KU060, proposing the first implementation of the considered case study on space qualified technology. We characterized the system in terms of resource utilization, performance, and power consumption and we provide a comparison with an Nvidia Jetson Nano, an embedded GPU commonly used for the processing of RNNs. The results show that our solution achieves lower inference time and power consumption, thus leading to better energy efficiency, but at the cost of a higher mean square error.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 10,2026 at 19:22:59 UTC from IEEE Xplore. Restrictions apply.

ACKNOWLEDGEMENTS

This work is partially supported by the Italian Ministry of Education and Research (MUR) in the framework of the FoReLab project (Departments of Excellence) and by the European Space Agency (ESA) in the framework of the Open Space Innovation Platform (OSIP) research activity (Contract No. 4000141108).

REFERENCES

[1] Vivek Kothari, Edgar Liberis, and Nicholas D. Lane. The final frontier: Deep learning in space. In *Proceedings of the 21st International Workshop on Mobile Computing Systems and Applications*, HotMobile '20, page 45–49, New York, NY, USA, 2020. Association for Computing Machinery.

[2] Xueyuan Dai, Xiaofeng Wu, Bin Wang, and Liming Zhang. Semisupervised scene classification for remote sensing images: A method based on convolutional neural networks and ensemble learning. *IEEE Geoscience and Remote Sensing Letters*, 16(6):869–873, 2019.

[3] Yang-Yang Zheng, Jian-Lei Kong, Xue-Bo Jin, Xiao-Yi Wang, Ting-Li Su, and Min Zuo. Cropdeep: The crop vision dataset for deep-learning-based classification and detection in precision agriculture. *Sensors*, 19(5), 2019.

[4] Yang-Yang Zheng, Jian-Lei Kong, Xue-Bo Jin, Xiao-Yi Wang, Ting-Li Su, and Min Zuo. Cropdeep: The crop vision dataset for deep-learning-based classification and detection in precision agriculture. *Sensors*, 19(5):1058, 2019.

[5] El Hadrami Cheikh Tourad and Mohsine Eleuldj. Survey of deep learning neural networks implementation on fpgas. In *2020 5th International Conference on Cloud Computing and Artificial Intelligence: Technologies and Applications (CloudTech)*, pages 1–8, 2020.

[6] Teng Wang, Chao Wang, Xuehai Zhou, and Huaping Chen. An overview of fpga based deep learning accelerators: Challenges and opportunities. In *2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, pages 1674–1681, 2019.

[7] Stylianos I. Venieris, Alexandros Kouris, and Christos-Savvas Bouganis. Toolflows for mapping convolutional neural networks on fpgas: A survey and future directions. *ACM Comput. Surv.*, 51(3), jun 2018.

[8] T. Pacini, E. Rapuano, and L. Fanucci. Fpg-ai: a technology-independent framework for the automation of cnn deployment on fpgas. *IEEE Access*, pages 1–1, 2023.

[9] Stefano Silvestrini and Michele R Lavagna. Spacecraft formation relative trajectories identification for collision-free maneuvers using neural-reconstructed dynamics. In *AIAA Scitech 2020 Forum*, page 1918, 2020.

[10] Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Soderstrom. Detecting spacecraft anomalies using lstms and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '18, page 387–395, New York, NY, USA, 2018. Association for Computing Machinery.

[11] Roberto Furfaro, Ilaria Bloise, Marcello Orlandelli, Pierluigi Di Lizia, Francesco Topputo, Richard Linares, et al. Deep learning for autonomous lunar landing. *Advances in the Astronautical Sciences*, 167:3285–3306, 2018.

[12] Emilio Rapuano, Tommaso Pacini, and Luca Fanucci. A post-training quantization method for the design of fixed-point-based fpga/asic hardware accelerators for lstm/gru algorithms. *Computational Intelligence and Neuroscience*, 2022, 2022.

[13] Nicola Ferrante, Gianluca Giuffrida, Pietro Nannipieri, Alessio Bechini, and Luca Fanucci. Fault detection exploiting artificial intelligence in satellite systems. In *2nd International Conference on Applied Intelligence and Informatics*, 2022.

Authorized licensed use limited to: VIT University- Chennai Campus. Downloaded on October 10,2026 at 19:22:59 UTC from IEEE Xplore. Restrictions apply.